Big-O Intuition

Why Some Lookups Stay Fast

```
names = ["Ana", "Bo", "Cy"]
print("Cy" in names) # scan
book = {"Cy": "555-0101"}
print(book["Cy"])  # jump
```

By the End of Today, You'll Be Able To…

1

Explain how search cost changes as data grows

2

Compare linear scan with dict lookup

3

Use Big-O language to defend a choice

You'll Know These by the End of Today

O(n)

Work grows with the number of items.

O(1)

Work stays roughly constant as data grows.

Reading Recap: When the Crowd Grows

Today's Reading: Big-O Intuition

An airport check-in screen finds your booking by its code. Reopen the reading:
When the Crowd Grows: Big-O Intuition

The Scenario: An Airport Check-In Screen

You type a 6-letter code. The screen finds your booking.

200 bookings: checking them one by one still feels instant.

200,000 bookings: one by one can mean 200,000 checks. You wait.

READING RECAP

Key Terms and Code From the Reading

LINEAR SEARCH

`"QX7RT2" in bookings` checks from the start, one by one.

INPUT SIZE, n

`len(bookings)` gives n, the number of items.

LOOKUP

`bookings[3]` jumps straight to the fourth item.

O(n) VS. O(1)

Work grows with n, or stays about flat.

READING CHECK

A gate scanner uses `in` on a 100-seat passenger list. Planes grow to 400 seats. Worst case?

A

Stays at 100 checks

B

About 1,600 checks

C

About 400 checks

ANSWER KEY

Answer Key: Bigger Planes, Bigger Scan

Correct answer: C. About 400 checks

`in` checks the list one passenger at a time. 4 times the seats means about 4 times the worst-case checks: O(n).

READING CHECK

Which one is a lookup, not a linear search?

A

```
bookings[3]
```

B

```
"QX7RT2" in
bookings
```

C

A `for` loop that checks each code

ANSWER KEY

Answer Key: Lookup vs. Linear Search

Correct answer: A. `bookings[3]`

`bookings[3]` jumps straight to one position: O(1). The other two check items in order from the start: O(n).

Your Questions From the Reading

Turn and talk for 1 min. Bring one question.

We take 2 or 3 questions now.

Bigger ones wait for today's matching slide.

Checking One at a Time

What Happens at Scale?

Every function so far has checked things one at a time — that habit works fine on 10 items. Does it still work fine on 10 million?

Why Does "Find One Item" Sometimes Take Forever?

1

Some searches always feel instant

Why Does "Find One Item" Sometimes Take Forever?

1

Some searches always feel instant

2

Some searches slow down as things grow

Why Does "Find One Item" Sometimes Take Forever?

1

Some searches always feel instant

2

Some searches slow down as things grow

3

Today's question: What makes the difference?

IN THE NEWS

Colliding Keys Made Fast Dictionary Lookups Crawl

100%

CPU, for up to several hours, from little attacker traffic

Figure from oCERT advisory 2011-003

Attackers sent forms full of colliding keys
Dicts searched those keys one by one
Servers could hit 100% CPU for hours

A dict jumps straight to a key. What would happen if it had to check keys one by one?

[Source: Advisory oCERT-2011-003, December 28, 2011]

Typing a Name vs. Scrolling Every Contact

1

Type a few letters, jump straight there

2

Scroll every name from A to Z

3

Same phone, same contact, very different effort

Analogy: Finding a contact by typing their name is like a dict lookup: Your phone jumps straight to the match, no matter how many thousand contacts you have. Finding it by scrolling from the top is not efficient

QUICK CHALLENGE

Quick check: In the contacts example, which one is more like checking a list item by item?

A

Typing a few letters to jump to the contact

B

Scrolling from A to Z checking every name

ANSWER KEY

Answer Key: Contacts Analogy Recall

Correct answer: Scrolling from A to Z checking every name

Scrolling checks one contact at a time, in order — the same shape as a linear scan through a list.

Scanning `[15, 47, 92, 8]` for 8

Linear Scan

- Checks items one at a time
- Gets slower as data grows

Direct Lookup

- Jumps straight to the match
- Stays fast no matter the size

In Common

- Both are real, valid choices

Watch It Run: A Linear Scan

Step 1 of 5

The list holds four numbers

```
numbers = [15, 47, 92, 8]
for n in numbers:
    print("Check", n)
    if n == 8:
        print("Found it")
```

OUTPUT

ANIMATED IN CLASS: watch it run in the lecture recording

How the Search Cost Grows

Key idea: **Linear scan grows with the data. Direct lookup stays roughly flat.**

Linear scan

- Checks one item at a time
- More items means more possible checks
- Big-O name: O(n)

Direct lookup

- Jumps straight to the key
- More items barely changes the work
- Big-O name: O(1)

That's the whole intuition behind "stays fast."

QUICK CHALLENGE

Quick math: A list has 500 items and the item you want is last. Worst case, how many checks does a linear scan take?

A

1

B

250

C

500

ANSWER KEY

Answer Key: Linear Scan Worst-Case Math

Correct answer: 500

Worst case, a linear scan checks every single item before reaching the last one — no shortcuts, one at a time.

The Pattern

This Has a Name

Engineers don't say "it depends on the list size" every time — there's a short way to say exactly this.

This Has a Name: O(1) and O(n)

O(n)

Work grows with the number of items.

O(1)

Work stays roughly constant as data grows.

QUICK CHALLENGE

Match it: Which term describes "the cost stays the same no matter how much data there is"?

A

O(1)

B

O(n)

C

Big-O

ANSWER KEY

Answer Key: Match the Term: O(1)

Correct answer: O(1)

O(1) is constant time — the exact definition just given, and the same thing the dict-lookup column in the table showed.

QUICK CHALLENGE

Quick recall: If you double the number of items in a list, what happens to a linear scan's worst-case cost?

A

It roughly doubles too

B

It stays about the same

C

It roughly halves

ANSWER KEY

Answer Key: O(n) Scaling Recall

Correct answer: It roughly doubles too

That's exactly what O(n) means — the cost scales directly with the size of the data.

QUICK CHALLENGE

Quick math: A linear scan takes at most 1,000 checks on a 1,000-item list. The list grows to 2,000 items — what's the new worst-case check count?

A

1,000

B

2,000

C

4,000

ANSWER KEY

Answer Key: Doubling the List

Correct answer: 2,000

O(n) scales directly with size — double the items, double the worst-case checks. No shortcuts.

TIP

“What Would You Change for 10,000 Lookups a Day?”

Scenario: 10,000 lookups a day in a customer list that keeps growing.

Answer with QAR: your first question, your assumption, the risk.

MODEL ANSWER

Model Answer: QAR

QUESTION

What do we look customers up by: ID, or name?

ASSUMPTION

By customer ID, and every ID is unique.

RISK

Duplicate IDs: A dict keeps only the last one.

QUICK CHALLENGE

Debug the reasoning: A Fellow says "O(n) means slow." What's missing from that claim?

A

Nothing — that's correct

B

O(n) only applies to dictionaries, not lists

C

O(n) describes how cost scales, not a fixed speed

ANSWER KEY

Answer Key: O(n) Misconception Check

Correct answer: O(n) describes how cost scales, not a fixed speed

An O(n) operation on a tiny list can be very fast in real seconds — the label describes the growth pattern as data scales up, not a verdict on any single run.

QUICK CHALLENGE

Which of these stays O(1) even at 10 million items: checking if a key exists in a dict, or scanning a list for its largest value?

A

Checking if a key exists in a dict

B

Scanning a list for its largest value

C

Both stay O(1)

ANSWER KEY

Answer Key: O(1) or Not: Two Operations

Correct answer: Checking if a key exists in a dict

A dict key check jumps straight to the slot — O(1). Finding the largest value means checking every single item at least once, no matter how the list is organized — that's O(n), not O(1).

Why This Matters for Your Future Job

A common early interview question

Real systems hit this at real scale

Defend a choice, don't just define it

TIP

“Explain Big-O to Someone Who's Never Coded.”

Answer like an interview, with PREP: point, reason, example, point, in plain language.

MODEL ANSWER

Model Answer: PREP

POINT

Big-O is how much more work a task takes as it grows.

REASON

It compares approaches by growth, not by exact seconds.

EXAMPLE

Scrolling every contact is O(n); typing the name jumps there, O(1).

POINT, RESTATED

Big-O tells you what happens when the data gets bigger?

Key Terms — Recap

Big-O

Describes how an algorithm's work grows as the input grows.

O(n)

Linear time; the work grows with the number of items.

O(1)

Constant time; the work stays roughly the same regardless of input size.

Linear scan

Checks items one at a time until the target is found.

So Far, We've Covered...

1

Why some lookups stay fast

2

This feeds the project's own reflection

3

Next: Real Python syntax for this idea

Before Next Class

- **No new exercise today, just the concept**
- Next class: Python data structures, with code